

nosh: A Compact Fine-Tuned Language Model for Natural-Language-to-Bash Command Generation on Consumer Hardware

Author Name
Independent Research

confusedstudent13@gmail.com

Abstract—Command-line interfaces remain a barrier for non-expert users, yet deploying large language models (LLMs) to translate natural language into shell commands is impractical on personal machines. We present **nosh**, a 1.5-billion parameter model that converts natural-language requests into executable Bash commands and runs comfortably on a commodity laptop with no GPU. Starting from Qwen2.5-Coder-1.5B-Instruct, we apply parameter-efficient LoRA fine-tuning and show that the dominant factor in command-generation quality is the *difficulty distribution* of the training data rather than the fine-tuning hyperparameters: an initial model trained on short, tool-homogeneous corpora is statistically indistinguishable from the base model on complex commands (4.18 vs. 3.50 mean judge score is the gap we later close), whereas adding a corpus of long, multi-tool pipelines raises the mean LLM-judge score on a held-out complex set from 3.50 to 4.18 (out of 5) and nearly doubles exact-match accuracy from 0.26 to 0.46. We evaluate with a reference-aware LLM-as-judge rubric that scores tool choice, flag coverage, argument fidelity, constraint satisfaction, and safety, complemented by deterministic syntax and formatting checks. Finally, we conduct a deployment study over four quantization tiers and four few-shot prompt lengths, finding that a 4-bit (Q4_K_M, 940MB) model with a two-example prompt is the quality/footprint sweet spot, sustaining a peak resident memory of ≈ 1.7 GB and a cold-start latency of 1.4–1.6s per query on CPU. The resulting tool integrates with the shell so that generated commands are pre-filled on the next prompt line for one-keystroke execution.

Keywords—*natural language interfaces, code generation, large language models, parameter-efficient fine-tuning, quantization, LLM-as-judge, shell commands*

I. INTRODUCTION

The Unix shell is among the most powerful interfaces in computing, yet its utility is gated by the user’s ability to recall exact command syntax, flags, and pipeline idioms. A user who knows *what* they want—“find every Python file changed in the last day and count the total lines”—may not recall that this maps to `find . -name '*.py' -mtime -1 -exec cat {} + | wc -l`. Natural-language-to-command translation promises to remove this barrier, but a practical assistant must satisfy three constraints simultaneously: it must produce *correct and complete* commands including the flags that encode user intent, it must run *locally* so that potentially sensitive requests never leave the machine, and it must be *fast and lightweight* enough to feel native inside an interactive shell. Cloud-scale LLMs satisfy the first constraint but violate the

latter two.

This paper presents **nosh**, a natural-language-to-Bash assistant built on a 1.5B-parameter model that runs entirely on consumer hardware. Our central finding is methodological rather than architectural. A first fine-tuned model, trained with reasonable hyperparameters on the corpora most readily available to us—short curated command pairs and a large but tool-homogeneous natural-language-to-Bash dataset dominated by `find`—taught the model to comply with output formatting but did *not* improve command correctness on complex requests: on a held-out set of 600 complex commands it scored 3.498 against the base model’s 3.503 on a five-point scale, a difference within noise. The model still hallucinated flags and arguments on anything beyond simple commands. Diagnosing this, we found the training distribution never contained the long, multi-tool pipelines the model was failing on.

Correcting the *data* rather than the optimization recipe produced the gain. Adding a corpus of complex, well-formed commands (average length 62 characters, 28% containing pipes) lifted the mean judge score on the same held-out set to 4.178, increased the fraction of commands rated 4 or 5 from 0.53 to 0.74, and nearly doubled exact-match accuracy from 0.26 to 0.46. This supports a broader claim: for small code-generation models, when a fine-tune fails to move a capability, the first suspect should be whether the training distribution covers that capability’s difficulty band, not the learning rate or adapter rank.

Beyond quality, we treat deployability as a first-class research question. We systematically study the trade-off between model footprint and generation quality across four GGUF quantization tiers, and separately measure how many in-context examples a fine-tuned model still needs. Both studies use the same LLM-as-judge evaluation as our main experiments, so deployment decisions are grounded in the same quality metric rather than proxy measures.

Our contributions are:

- A demonstration, with controlled before/after evaluation, that training-data *difficulty coverage*—not fine-tuning hyperparameters—is the binding constraint on complex-command quality for a 1.5B model, closing a +0.68 mean-judge-score gap on held-out complex commands.
- A reference-aware LLM-as-judge rubric for command

generation that assigns partial credit across tool, flags, arguments, constraints, and safety, paired with deterministic syntax and formatting pre-checks.

- A deployment study over quantization tiers and few-shot lengths that identifies a 940 MB, two-shot configuration running in ≈ 1.7 GB of RAM at 1.4–1.6 s latency on CPU, integrated into the shell for one-keystroke execution.

II. RELATED WORK

Natural language to shell commands. Translating English to shell commands was framed as a semantic-parsing and neural machine-translation problem by the NL2Bash effort of Lin *et al.* [1], which released a corpus of English-to-Bash pairs and established token- and template-level metrics. Earlier interactive systems such as Tellina explored neural translation for file-system operations. These systems predate instruction-tuned LLMs and generally target a narrower command vocabulary than modern code models; we reuse the NL2Bash data as one component of our training mixture but find, consistent with our central result, that its heavy skew toward `find` limits its value for complex multi-tool commands.

Code language models. General code models such as Code Llama [2] and the Qwen2.5-Coder series [3] are pre-trained on large code corpora and exhibit strong zero-shot shell competence, but at parameter counts and memory footprints that preclude comfortable local, CPU-only deployment. We build on the smallest instruction-tuned Qwen2.5-Coder checkpoint precisely to keep the deployed footprint within a laptop’s means.

Parameter-efficient fine-tuning. Low-Rank Adaptation (LoRA) [4] freezes the base weights and learns low-rank update matrices, reducing the trainable-parameter count by orders of magnitude and making single-GPU fine-tuning of billion-parameter models tractable. We use LoRA throughout and merge the adapter into the base weights before quantization.

Quantization for local inference. Post-training quantization to 3–5 bits via the GGUF/llama.cpp k -quant formats [6] enables CPU inference of billion-parameter models within gigabyte-scale memory budgets. Prior work reports quality/size trade-offs at the level of perplexity; we instead measure the trade-off directly in task quality using our command-generation judge.

LLM-as-judge evaluation. Using a strong LLM to score the outputs of another model correlates well with human preference and scales to large evaluation sets [5]. Exact-match against a single reference badly underestimates command-generation quality because many distinct commands are functionally equivalent (e.g. `ls -la` vs. `ls -al`). We adopt a reference-aware judge that treats the dataset command as a hint about intent rather than the sole correct answer.

III. SYSTEM DESIGN

A. Overview

nosh comprises three stages: a data-preparation pipeline that assembles a difficulty-balanced training mixture, a LoRA fine-tuning stage over Qwen2.5-Coder-1.5B-Instruct, and a

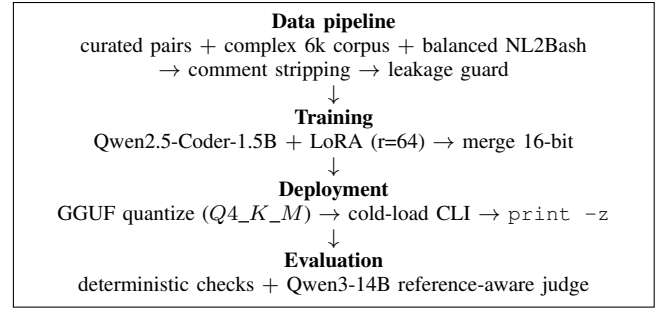


Fig. 1. The nosh pipeline. Data assembly, parameter-efficient fine-tuning, quantized deployment, and LLM-as-judge evaluation are decoupled stages.

quantized local-inference command-line application. Evaluation is performed by a separate LLM-as-judge stage that scores generated commands against a multi-dimensional rubric. Fig. 1 summarizes the flow.

B. Data Preparation

The training mixture is assembled from three sources with deliberately different difficulty profiles. The first is a set of curated, domain-focused natural-language/command pairs emphasizing developer and system-administration tools (`git`, `docker`, `kubectl`, `sudo`). The second is a 5,823-example corpus of complex, well-formed commands whose average length is 62 characters and 28% of which contain pipes—the difficulty band the model most needed. The third is the NL2Bash corpus [1], which is large but heavily skewed: roughly 60% of its commands invoke `find`. We counter this skew by capping the number of examples per leading command token and sampling a balanced subset.

Two cleaning steps proved important. First, many complex-corpus entries append explanatory comment lines (e.g. `# [output of first 10 lines]`) beneath the real command; training on these would teach the model to emit prose, in direct violation of the “output only a command” contract, so we strip pure-comment lines while preserving genuine `#!` shebang scripts. Second, we apply an explicit leakage guard that removes any training row whose natural-language prompt appears in any evaluation set. The resulting training set contains 9,086 deduplicated examples spanning 890 distinct leading command tokens with a mean command length of 49.9 characters. We hold out 600 complex examples (TEST-6K) plus two curated 100-example sets (009, 010) with zero prompt overlap into training.

C. Model and Fine-Tuning

We fine-tune Qwen2.5-Coder-1.5B-Instruct [3] with LoRA [4] applied to all attention and MLP projection matrices. The final model (V2) uses rank 64 and scaling $\alpha = 64$, trained for two epochs with an effective batch size of 32 (micro-batch 8, gradient accumulation 4), a peak learning rate of 2×10^{-4} under a cosine schedule, and 8-bit optimizer states. Training runs on a single 24 GB consumer GPU. Loss during in-training generation previews at full batch exceeded memory; disabling previews and halving the micro-batch (with compensating accumulation) resolved this without changing the effective batch size. After training, the LoRA adapter is merged into the base weights in a separate

process to produce a standalone 16-bit checkpoint suitable for quantization.

Prompts follow a fixed system instruction that constrains the model to emit only a valid Bash command with no markdown, prose, or code fences, optionally preceded by a small number of few-shot demonstrations. The same prompt shape is used at training and inference time.

D. Evaluation Rubric

Because exact match against a single reference command underestimates quality, we score generations with a reference-aware LLM-as-judge [5]. The judge receives the instruction, the generated command, a reference command (explicitly labeled a hint rather than ground truth), and two pre-computed deterministic signals. It returns partial-credit scores along five dimensions: tool appropriateness (boolean), flag coverage (0–2), argument fidelity (0–2), constraint satisfaction (0–2; e.g. “read-only”, “exclude node_modules”, a specific codec), safety (boolean; penalizing unrequested destruction such as `rm -rf /`), and a holistic overall score (1–5). Two deterministic pre-checks feed the judge and are also reported independently: a formatting check that rejects markdown, JSON, or conversational preambles, and a syntax check that runs `bash -n` to confirm the command parses.

IV. EXPERIMENTAL SETUP

Models compared. We compare three systems: the unmodified Qwen2.5-Coder-1.5B-Instruct *baseline*; *V1*, a LoRA fine-tune trained only on the curated pairs plus balanced NL2Bash; and *V2*, the final fine-tune that adds the complex-command corpus. V1 and V2 differ in both the training mixture and the adapter rank (32 vs. 64); the ablation of interest is the mixture, and we discuss this confound in Section VI.

Evaluation sets. TEST-6K is the 600-example held-out slice of the complex corpus and is our primary benchmark. 009 and 010 are 100-example curated held-out sets used as secondary checks. No evaluation prompt appears in training.

Judge. All judgments use Qwen3-14B (4-bit) as the scorer, run under vLLM with greedy decoding for determinism. Deterministic checks (`bash -n`, formatting) are computed in-process.

Metrics. We report the mean overall judge score (OVERALL, 1–5), the fraction of commands rated at least 4 (PCT $\geq$ 4), normalized exact-match against the reference (EXACT), and the `bash -n` pass rate (SYNTAX).

V. RESULTS

A. Data Difficulty Drives Complex-Command Quality

Table I reports the three systems across all evaluation sets. On the primary TEST-6K benchmark, V1 is statistically indistinguishable from the untuned baseline (3.498 vs. 3.503 overall), despite being a fine-tuned model: it learned the output format—its formatting-compliance rate is 0.998—but not complex-command competence. V2, which differs from V1 chiefly by the inclusion of the complex-command corpus, raises the overall score to 4.178, a +0.68 absolute gain,

TABLE I. MAIN RESULTS ACROSS EVALUATION SETS. TEST-6K IS THE PRIMARY HELD-OUT COMPLEX BENCHMARK ($n=600$); 009/010 ARE CURATED HELD-OUT SETS ($n=100$). JUDGE: QWEN3-14B. BEST PER COLUMN IN BOLD.

Set	Model	OVERALL	PCT $\geq$ 4	EXACT	SYNTAX
TEST-6K	Baseline	3.503	0.533	0.257	0.973
	V1	3.498	0.548	0.230	0.968
	V2	4.178	0.740	0.455	0.990
009	Baseline	3.640	0.560	0.230	0.920
	V1	3.830	0.630	0.290	1.000
	V2	3.990	0.680	0.320	1.000
010	V2	3.970	0.660	0.320	1.000

TABLE II. FEW-SHOT LENGTH ABLATION ($Q4_K_M$, $n=250$). SCORES ARE FLAT WITHIN NOISE; THE FINE-TUNE HAS INTERNALIZED THE TASK.

# example pairs	OVERALL	PCT $\geq$ 4	EXACT
0	4.036	0.716	0.388
2	4.040	0.712	0.404
4	4.104	0.716	0.404
8	4.060	0.708	0.404

lifts PCT $\geq$ 4 from 0.53 to 0.74, and nearly doubles exact-match from 0.26 to 0.46. The per-dimension scores move consistently: argument fidelity rises from 1.45 to 1.71 and constraint satisfaction from 1.26 to 1.61 (both out of 2), indicating the gain is concentrated exactly where V1 failed—in reproducing the specific flags, values, and constraints of hard requests.

The curated 009/010 sets corroborate the trend at smaller magnitude: V2 reaches 3.99 and 3.97 overall, improving on both baseline (3.64 on 009) and V1 (3.83). The smaller gap here is expected, as these sets contain shorter commands closer to V1’s training distribution. The `bash -n` syntax pass rate reaches 0.99–1.00 for V2 across all sets, confirming the model reliably emits parseable commands.

B. Few-Shot Examples Are Largely Redundant After Fine-Tuning

Because the deployed application re-encodes any few-shot prefix on every invocation, we ask how many demonstrations the fine-tuned model still requires. Table II varies the number of in-context example pairs from 0 to 8 at fixed quantization ($Q4_K_M$) on a 250-example subset. The overall score is flat within noise across all settings (4.04 to 4.10), and even zero-shot performance (4.036) is competitive. Fine-tuning has internalized the task and the output contract, so few-shot prompting is nearly redundant. We retain two examples in the default configuration for a marginal exact-match benefit (0.404 vs. 0.388 at zero-shot) and output-format anchoring at negligible cost.

C. Quantization Trade-off

Table III evaluates four GGUF k -quant tiers at two-shot prompting on a 100-example subset. Quality is non-monotonic in bit-width: $Q4_K_M$ (940 MB) attains the highest overall score (4.13), while the larger $Q5_K_M$ (1073 MB) is no better (3.99), consistent with quantization noise dominating the small remaining headroom. The more aggressive $Q3_K_M$

TABLE III. QUANTIZATION TIER TRADE-OFF (TWO-SHOT, $n=100$). $Q4_K_M$ IS THE QUALITY MAXIMUM; LARGER TIERS ADD MEMORY WITHOUT GAIN.

Tier	Size (MB)	OVERALL	PCT ≥ 4	EXACT	SYNTAX
$Q3_K_M$	786	3.94	0.67	0.33	0.99
$Q4_K_S$	897	4.05	0.67	0.39	0.99
$Q4_K_M$	940	4.13	0.72	0.39	0.97
$Q5_K_M$	1073	3.99	0.70	0.40	0.99

(786MB) loses roughly 0.2 overall and was additionally observed to emit occasional non-UTF-8 bytes, a qualitative reliability concern. $Q4_K_M$ is thus both the quality maximum and near the footprint minimum: no tier offers a meaningful memory saving without a quality regression. We adopt $Q4_K_M$ as the deployed configuration.

D. Deployment Characteristics

The final application cold-loads the 940MB quantized model on each invocation—no persistent daemon—to keep the tool stateless and simple. On a CPU-only configuration, representative of a laptop without a discrete GPU, end-to-end latency including model load and generation is 1.4–1.6s per query and peak resident memory is ≈ 1.7 GB (model weights plus runtime and key/value cache buffers at a 512-token context). This footprint fits comfortably within an 8GB machine. The context window is capped at 512 tokens, well above the observed prompt-plus-command length, which bounds the key/value cache and reduces compute. The generated command is emitted on standard output and injected into the shell’s next prompt line (via `print -z` in Zsh), so the user reviews and executes it with a single keystroke.

E. Qualitative Examples

The deployed model produces idiomatic pipelines for requests well outside the literal training strings. Representative outputs include:

```
NL : show the top 10 most memory-hungry processes
CMD: ps aux --sort=-%mem | head -n 11
```

```
NL : count unique IP addresses in access.log
      sorted by frequency descending
CMD: awk '{print $1}' access.log | sort
      | uniq -c | sort -nr
```

```
NL : kill the process listening on port 8080
CMD: kill $(lsof -t -i:8080)
```

These exhibit correct multi-stage composition, appropriate flag selection, and command substitution—capabilities absent from the V1 model.

VI. DISCUSSION AND LIMITATIONS

Mixture/rank confound. V1 and V2 differ in both training mixture and adapter rank (32 vs. 64). We attribute the gain to the mixture because the improvement is concentrated on the complex held-out set and in the argument and constraint dimensions that correspond directly to the added data, whereas rank alone would be expected to help more uniformly. A rank-controlled ablation (training rank 64 on the V1 mixture) would isolate the effect fully and is left to future work.

Judge as oracle. Our primary metric is a single LLM judge. While reference-aware LLM judging correlates well with human preference [5], it can share blind spots with the generator and does not execute commands. Sandbox execution-based evaluation would provide a complementary, harder-to-game signal.

Subset sizes in the deployment study. The quantization and few-shot sweeps use 100- and 250-example subsets; sub-0.1 score differences between adjacent configurations are within sampling noise. The robust conclusions are directional—“ $Q4_K_M$ is the sweet spot and lower tiers regress,” “few-shot examples are largely redundant”—rather than precise rankings among statistically tied configurations.

Safety. The safety dimension penalizes unrequested destructive operations at judging time, but nosh does not execute commands and inserts them for user confirmation rather than auto-running them. Human-in-the-loop review remains the primary safety mechanism.

VII. CONCLUSION

We presented nosh, a 1.5B-parameter natural-language-to-Bash assistant that runs on consumer hardware within ≈ 1.7 GB of memory. Our main finding is that for small code-generation models, the difficulty coverage of the training mixture—not the fine-tuning hyperparameters—was the binding constraint on complex-command quality: adding a corpus of long, multi-tool commands raised the mean LLM-judge score on held-out complex commands from 3.50 to 4.18 and nearly doubled exact-match accuracy, whereas an earlier fine-tune lacking that data matched the untuned baseline. We further showed, using the same task-quality metric, that a fine-tuned model needs almost no few-shot examples and that 4-bit $Q4_K_M$ quantization is the quality/footprint sweet spot for local deployment. Future work includes rank-controlled ablations to fully isolate the data effect, execution-based evaluation in a sandbox, and extending the difficulty-coverage principle to other structured-output generation tasks.

REFERENCES

- [1] X. V. Lin, C. Wang, L. Zettlemoyer, and M. D. Ernst, “NL2Bash: A Corpus and Semantic Parser for Natural Language Interface to the Linux Operating System,” in *Proc. 11th Int. Conf. Language Resources and Evaluation (LREC)*, 2018.
- [2] B. Rozière *et al.*, “Code Llama: Open Foundation Models for Code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [3] B. Hui *et al.*, “Qwen2.5-Coder Technical Report,” *arXiv preprint arXiv:2409.12186*, 2024.
- [4] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
- [5] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [6] G. Gerganov *et al.*, “llama.cpp: LLM inference in C/C++,” 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>